MAKESPAN OPTIMIZATION IN SOFTWARE PROJECTS: BEHAVIORAL AND TECHNICAL ALIGNMENT

Roberto Júnior Cassimiro Campolina
Operations Research and Complex Systems Lab (ORCS)
Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brazil
robertojuniorme@gmail.com

André Costa Batista
Departamento de Engenharia Elétrica
Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brazil
andre-costa@ufmg.br

ABSTRACT

The management of development activities in software projects can be a complex tactical challenge, where the allocation of tasks to human resources can become a bottleneck. Task allocation in software projects is traditionally modeled considering only technical constraints. This paper proposes a Mixed-Integer Linear Programming (MILP) model for the Software Project Scheduling Problem (SPSP) that incorporates behavioral compatibility. The formulation penalizes misalignment through a cubic polynomial function of Solides profiles, allied with a technical bonus. Solving 1380 instances in the Gurobi Optimizer demonstrates that cognitive noise increases the makespan and mathematically validates Brooks's Law, providing a relevant metric for the project's tactical planning.

KEYWORDS. Operations Research. Mixed-Integer Programming. Software Engineering. Agile Management. SPSP.

Paper topics: OR in Industry. Project Management. Combinatorial Optimization.

1. Introduction

The software development industry has consolidated itself as one of the main pillars of the modern global economy, characterized by rapid delivery cycles and dynamic requirements. Unlike traditional manufacturing environments, where machine processing is deterministic, the software life cycle is fundamentally dependent on human capital. Efficiently managing these resources is a challenge in software factories, where professional allocation and architectural code dependencies affect the project schedule [Dikert et al., 2016; Faria et al., 2022].

In Operations Research literature, this problem is formalized as the Software Project Scheduling Problem (SPSP), a specialized variant of the classic *Resource-Constrained Project Scheduling Problem* (RCPSP). Given the combinatorial nature of task assignments under precedence and capacity constraints, the SPSP is classified as strongly NP-Hard [Blazewicz et al., 1983; Kolisch and Padman, 1999]. A comprehensive review of SPSP, detailing the different approaches and resolution methods explored in the literature, can be found in the survey by Vega-Velázquez et al. [2018].

Mathematical models for SPSP often differentiate teams strictly by their technical skills [Alba and Chicano, 2007; Chicano et al., 2011; Pritsker et al., 1969]. This simplification does not account for cognitive and behavioral factors inherent to software engineering [DeMarco and Lister, 1987]. Furthermore, the non-linearity of productivity in software projects is documented in empirical literature. Brooks [1975] formulated Brooks's Law, which postulates that adding manpower to a late software project tends to make it even later, due to the exponential growth in communication *overhead*[1] and the restrictive topology of precedences. Although a recurring concept in agile management, this dynamic is rarely structured as an endogenous variable in analytical MILP models [Ryan and Cho, 2013; Acuña et al., 2006].

This paper aims to represent behavioral alignment through a numerical component within a mathematical optimization model. Using the Solides psychometric methodology, which categorizes professionals into Executors, Communicators, Planners, and Analysts [Costa and Lima, 2018], the model adjusts a task's nominal execution time based on the psychological compatibility and technical surplus of the individual. The study integrates behavioral profiles into an SPSP formulation, evaluates a cubic polynomial penalty function, analyzes the mathematical effects of Brooks's Law, and explores tactical capacity planning through a heatmap.

The remainder of this paper is organized as follows. Section 2 details the mathematical modeling and the time adjustment function. Section 3 describes the experimental methodology and the generation of computational instances. Section 4 presents the results obtained by the optimizer and the managerial analyses. Finally, Section 5 concludes the work and proposes guidelines for future research.

2. Mathematical Modeling

The central premise of this model is that technical productivity is impacted by the developer's profile. The proposed MILP formulation calculates an adjusted execution time *a priori* for each task-developer pair based on combined technical and behavioral characteristics.

[1]Refers to the time and effort consumed in exchanging information and aligning among team members. As the number of members in the same activity increases, the complexity of communication lines grows exponentially, generating progressive losses in net productivity.

2.1. Sets, Parameters, and Variables

Sets and Indices

M	Set of developers, indexed by $i \in \{1, \ldots, \lvert M \rvert\}$.
N	Set of tasks in the *backlog*, indexed by $j, k \in \{1, \ldots, \lvert N \rvert\}$.
S_{max}	Set of available *sprints*, indexed by $s \in \{1, \ldots, \lvert S_{max} \rvert\}$.
H	Set of technical skills, indexed by $h \in \{1, \ldots, \lvert H \rvert\}$.
P	Set of the 4 behavioral quadrants, indexed by $p \in \{1, \ldots, 4\}$.

Parameters

Ht_{jh}	Minimum technical level h required by task j.
Hd_{ih}	Current technical level of developer i in skill h.
Pt_{jp}	Fraction of behavioral profile p required for task j.
Pd_{ip}	Fraction of behavioral profile p of developer i.
T_j^{base}	Estimated base time to complete task j (hours).
Cap_{is}	Available capacity (hours) of developer i in *sprint* s.
δ_s	Duration of each *sprint* in calendar days.
j_{lab}	Usable working hours per day.
$Pred(j)$	Set of tasks that strictly precede task j.

Decision Variables

$x_{ijs} \in \{0, 1\}$	1 if task j is assigned to developer i in *sprint* s, 0 otherwise.
$y_s \in \{0, 1\}$	1 if *sprint* s is used, 0 otherwise.
$TF_s \geq 0$	Continuous variable of the accumulated final time (days) in *sprint* s.
$T^{max} \geq 0$	Continuous variable representing the project's *Makespan*.

2.2. Dynamic Time Calculation (T_{ij})

The numerical adjustment of the execution time is structured in three stages that quantify the impact of human variables on productivity:

Stage 1: Technical Surplus (ET_{ij})
Evaluates the surplus technical qualification of the assigned resource. If the developer exceeds the minimum stipulated requirement, task execution is accelerated. A scalar factor of 0.5 is incorporated into the equation as a damping coefficient, smoothing the productivity gain to be more consistent with practical reality:

$$ET_{ij} = 0.5 \cdot \left[\frac{1}{|H|} \sum_{h \in H} (\max(0, Hd_{ih} - Ht_{jh}))^2 \right] \quad \forall i \in M, \forall j \in N \tag{1}$$

Stage 2: Behavioral Error (EC_{ij})
To quantify the cognitive wear caused by psychological misalignment, the absolute difference between the quadrants of the behavioral profile required by the task and the developer's profile, denoted by ψ_{ij}, is initially calculated. The division by 2 acts as a normalization factor: since behavioral profiles are represented by proportional distributions (whose vector

sum is always 1.0), the maximum possible absolute difference between two diametrically opposed profiles results in 2.0. This division ensures that the metric ψ_{ij} strictly converges to the continuous interval of $[0, 1]$. Next, a polynomial function of order n with a gain multiplier k is applied to penalize more severe misalignments. These parameters were empirically defined as $n = 3$ and $k = 5.0$, a formulation whose justification is detailed in Section 4.1:

$$\psi_{ij} = \frac{1}{2} \sum_{p \in P} |Pd_{ip} - Pt_{jp}| \quad \forall i \in M, \forall j \in N \tag{2}$$

$$EC_{ij} = 5.0 \cdot (\psi_{ij})^3 \quad \forall i \in M, \forall j \in N \tag{3}$$

Stage 3: Adjusted Execution Time (T_{ij})
The final composition consolidates the simultaneous interaction of variables over the nominal time, assigning the efficiency gain provided by technical qualification and the penalty generated by behavioral inadequacy:

$$T_{ij} = T_j^{base} \times (1.0 + EC_{ij} - ET_{ij}) \quad \forall i \in M, \forall j \in N \tag{4}$$

2.3. Objective Function and Constraints

The model minimizes the final delivery schedule of the project, represented by the *makespan*:

$$\min T^{max} \tag{5}$$

The task allocation is subject to the operational and logical constraints described below. The first constraint ensures that each task is processed exactly once and assigned to a single developer in the project cycle:

$$\sum_{i \in M} \sum_{s \in S_{max}} x_{ijs} = 1 \quad \forall j \in N \tag{6}$$

The qualification constraint prevents assignment if the designated resource does not possess the minimum technical knowledge required by the task in all skills:

$$\sum_{i \in M} \sum_{s \in S_{max}} Hd_{ih} \cdot x_{ijs} \geq Ht_{jh} \quad \forall j \in N, \forall h \in H \tag{7}$$

The workload is limited by the capacity constraint, which restricts the total hours worked by a developer to their available limit in the respective *sprint*:

$$\sum_{j \in N} T_{ij} \cdot x_{ijs} \leq Cap_{is} \quad \forall i \in M, \forall s \in S_{max} \tag{8}$$

To preserve the code topology, the structural precedence constraint forces successor tasks to be scheduled in *sprints* strictly after their predecessors:

$$\sum_{i \in M} \sum_{s \in S_{max}} s \cdot x_{ijs} \geq \sum_{i \in M} \sum_{s \in S_{max}} s \cdot x_{iks} + 1 \quad \forall j \in N, \forall k \in Pred(j) \tag{9}$$

The *sprint* activation logic ensures that tasks are only allocated in active cycles and prevents the emergence of inactive gaps (holes) in the chronological planning:

$$\sum_{i \in M} \sum_{j \in N} x_{ijs} \geq y_s \quad \forall s \in S_{max} \tag{10}$$

$$x_{ijs} \leq y_s \quad \forall i \in M, \forall j \in N, \forall s \in S_{max} \tag{11}$$

$$y_{s-1} \geq y_s \quad \forall s \in S_{max} \setminus \{1\} \tag{12}$$

The allocated effort hours are converted into calendar days, structuring the continuous temporal progression of work throughout the *sprint*:

$$\delta_s \cdot (s-1) \cdot y_s + \frac{1}{j_{lab}} \sum_{j \in N} x_{ijs} \cdot T_{ij} \leq TF_s \quad \forall i \in M, \forall s \in S_{max} \tag{13}$$

Finally, the main variable of interest of the objective function is formalized, ensuring that the *makespan* encompasses the completion time of the last used *sprint*:

$$TF_s \leq T^{max} \quad \forall s \in S_{max} \tag{14}$$

3. Experimental Methodology

To guarantee statistical reliability, a Python generator instantiated the input matrices pseudo-randomly. The dataset comprises 1380 instances, ensuring 30 samples per scenario, with nominal times (T_j^{base}) projected for deadlines of 30 to 100 calendar days.

In all experiments, the team size (M) was restricted to the interval of 3 to 8 members. This range reflects the real standard for agile *squads*, given that the literature demonstrates that teams with more than 9 members suffer drastic productivity drops due to excessive communication and coordination Kniberg and Ivarsson [2012]; Schwaber and Sutherland [2017]. The instances were categorized into five groups:

- **MILP Scalability:** Examines the computational effort and combinatorial growth of the optimization model by progressively sizing the task *backlog* ($N \in [10, 80]$), keeping the team fixed at the median value of $M = 5$.
- **Behavioral Noise:** Evaluates the impact on the deadline under four allocations: *Baseline*, Natural, Critical, and Extreme.
- **Brooks's Law:** Analyzes tactical capacity planning by iterating the task volume ($N \in [25, 60]$) and the team size ($M \in [3, 8]$).
- **Precedence Density:** Quantifies how the proportion of interdependence between tasks impacts the total *makespan*, identifying the saturation point where the project becomes infeasible within the *sprint* budget.
- **Technical Surplus:** Evaluates the productivity gains of highly qualified teams in contrast to behavioral noise, identifying the break-even point between seniority and psychological alignment.

The formulations were processed through the Gurobi Optimizer, executed in a computational environment equipped with a 13th Gen Intel Core i7-1360P processor (2.20 GHz) and 16 GB of RAM. In order to provide a balanced trade-off between convergence quality and response time in instances with more robust dimensionality, the global maximum time limit was set to 3600 seconds (one hour), with a relative optimality tolerance (MIP Gap) established at 0.1%.

To ensure full reproducibility of the experiments and methodological transparency, the complete Python source code (including the MILP modeling, the instance generator, and the visual analysis scripts), as well as the raw output data, are publicly available in the project's official repository, accessible through the link: `https://github.com/rob1913r/behavioral-spsp`.

4. Results and Discussion

This section presents the computational results derived from solving the instances in the optimizer. The discussion begins with the evaluation of the theoretical basis of the adopted penalty function and the computational efficiency of the model across different task volumes. Following this, the data is explored to quantify the impact of behavioral noise on the *makespan*, mathematically validate the capacity constraints imposed by Brooks's Law, and investigate the empirical compensation relationship between technical surplus and psychological alignment.

4.1. Theoretical Modeling of the Behavioral Penalty

The choice of the cubic function for the Behavioral Error was based on the progressive analysis of misalignment. Figure 1 divides the error domain $\psi \in [0, 1]$ into the Acceptance (0.0 to 0.3), Critical (0.3 to 0.7), and Extreme (0.7 to 1.0) zones. It was noted that linear or quadratic functions punish small deviations excessively, while quartic functions underestimate moderate friction. The cubic function proved to be more realistic: it absorbs mild divergences and imposes steep penalties only upon reaching the critical range.

With the empirical multiplier $k = 5.0$ (Figure 2), the model reflects two realities: in the critical zone, time suffers severe inflations (up to 3× the nominal value), representing real bottlenecks in communication and rework. In the extreme zone, the penalty reaches the practical infeasibility threshold (6×), mirroring the breaking point where the task exceeds the *sprint's* capacity limit and forces the mathematical rejection of the assignment.

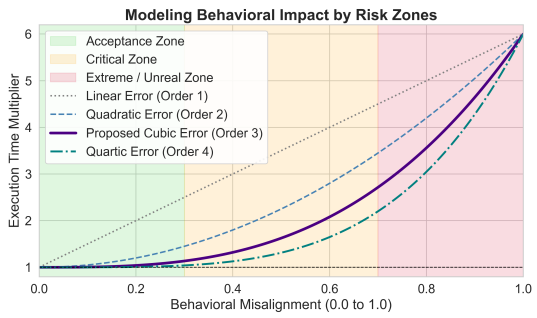

Figure 1: Zones of impact by polynomial function.

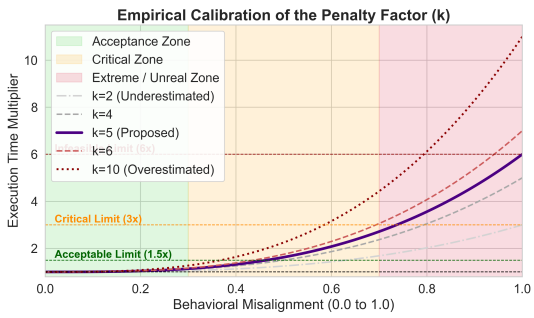

Figure 2: Calibration of the cubic multiplier k.

4.2. Scalability and Computational Efficiency of the Optimizer

The advancement of computational effort and its adherence to NP-Hard classifications can be clearly observed in light of the escalation of task volume (Figures 3 and 4) and the quantitative increase of active decision variables inside the solver (Figures 5 and 6).

For instances containing up to 40 independent tasks, the matrix converged in less than 5 seconds of processing. However, as complexity approached the $N = 80$ tasks mark, a sharp transition in the slope of the curve is noticed. The execution time required to prove the 0.1% optimality reached margins close to one minute in the worst formulations of the set, highlighting the increased depth of the *Branch and Bound* algorithm tree.

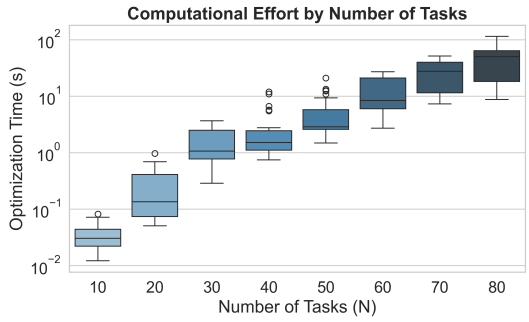

Figure 3: Computational effort by Tasks.

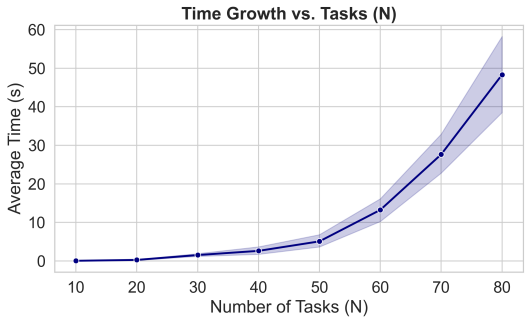

Figure 4: Time Growth vs. Tasks.

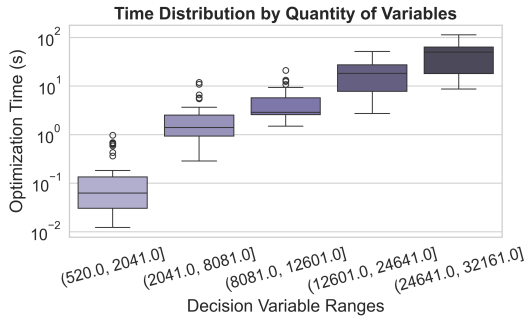

Figure 5: Time by quantity of Variables.

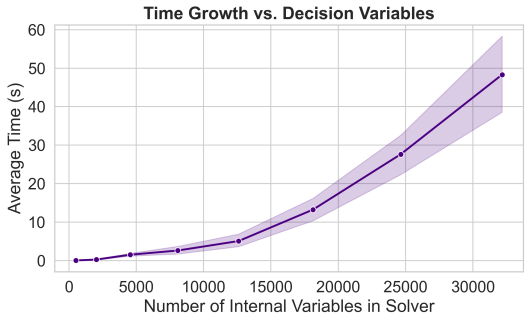

Figure 6: Time Growth vs. Variables.

4.3. Evolution under the Effect of Behavioral Noise

The *Baseline* scenario acts as a mathematical lower bound of time for the schedule. It represents an "ideal team", characterized by an abstraction where the developers' profiles are perfectly identical to the tasks' requirements ($Pd_{ip} = Pt_{jp}$), nullifying the behavioral error. According to Figure 7, the transition from this scenario to the *Natural* one (stochastic profiles) resulted in equivalent delivery deadlines. This occurs because the cubic penalty function absorbs subtle deviations within the Acceptance Zone ($\psi_{ij} \leq 0.3$). Consequently, the MILP algorithm manages to distribute the tasks so adherently that it nullifies the inherent noise of a real team, matching the efficiency of a perfect scenario.

However, there is a practical limit to this combinatorial compensation. In the *Critical* scenario, the scarcity of sufficiently compatible profiles removes the optimizer's evasion routes, inflating the *makespan* from an average of 28 to over 40 days. This degradation reaches its peak in the *Extreme* scenario, where allocations are forced to diametrically opposed profiles and deadlines nearly double, approaching 80 days (Figure 8). At this level, the incompatibility raises the task time (T_{ij}) so much that it exceeds the *sprint's* nominal hours limit. In order for the solver not to declare the project infeasible, the temporary relaxation of the capacity constraint (Cap_{is}) becomes essential.

The growth of the Behavioral Error (EC) responsible for this impact is detailed in Figure 9. The presence of the standard deviation bars validates the methodological design: they exist in the *Baseline* and *Natural* scenarios due to the stochastic generation (Dirichlet distribution), which simulates the organic variability of the teams. In contrast, these bars are null in the *Critical* and *Extreme* scenarios, as deterministic and fixed profiles were employed

in these to isolate and quantify the maximum impact of friction without the interference of randomness.

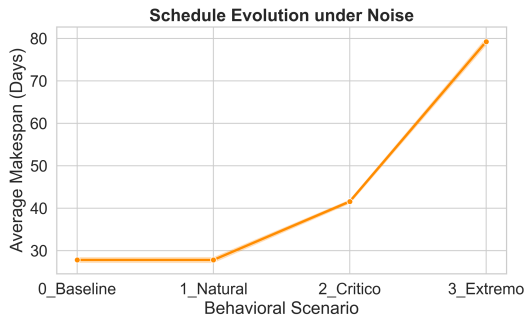

Figure 7: Schedule evolution under noise.

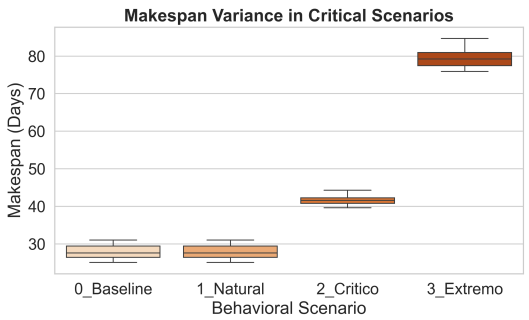

Figure 8: Variance in behavioral scenarios.

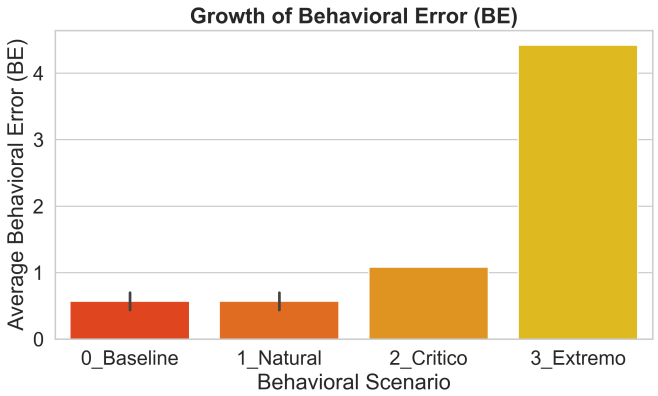

Figure 9: Growth of Behavioral Error (EC) by scenario.

4.4. Tactical Capacity Planning and Brooks's Law

From a managerial perspective, the model acts as a direct decision-support tool. The tactical heatmap (Figure 10) allows leaders to quickly evaluate the cost-benefit of resizing the team. In the case of $N = 50$ tasks, for example, expanding the team from 3 to 4 developers reduces the *makespan* from 92 to 66 days (a drop of $\approx 28\%$). On the other hand, jumping from 6 to 8 professionals brings a lower marginal return, reducing the deadline from 44 to 32 days ($\approx 27\%$ accumulated in the leap of two developers).

This exhaustion of efficiency, isolated in Figure 11 for the demand of $N = 50$ tasks, evidences Brooks's Law quantitatively. The law postulates that adding manpower to a project hits limits of parallelism and increased communication costs. The algorithm replicated this empirical phenomenon: from a certain team size, the time curve becomes an asymptote. This indicates to managers that project dependencies create a ceiling where adding more people stops generating significant gains in delivery speed.

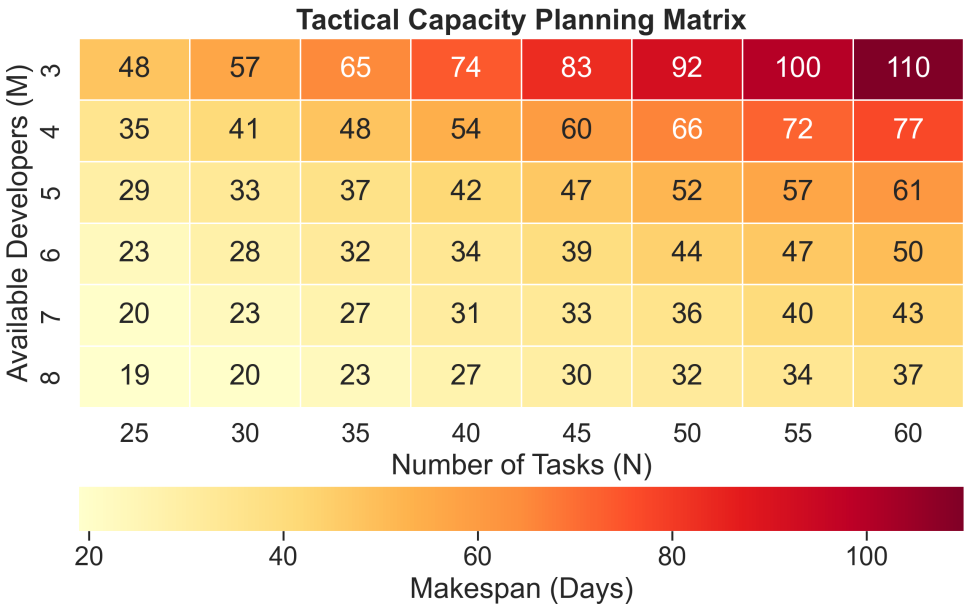

Figure 10: Tactical Capacity Planning Matrix.

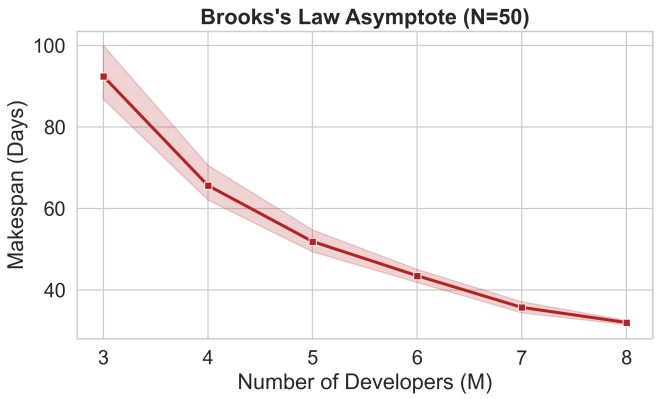

Figure 11: Brooks's Law Asymptote ($N = 50$).

4.5. Parallelism Limitations due to Precedence Density

The drop in efficiency observed in larger teams is directly linked to how the tasks in the backlog are interconnected. Graphs 12 and 13 show that excessive links and dependencies between activities limit the optimizer's ability to organize allocations in parallel.

When the proportion of precedence relations exceeds the 30% threshold of possible connections, delivery times begin to rise rapidly. More critical than the isolated delay is the saturation point identified in Graph 13. At this level of interdependence, the model reaches a limit where the volume of tasks can no longer be fitted within the available *sprint* budget, making the project mathematically infeasible. This scenario reinforces the current trend of replacing monolithic code structures with microservices. By reducing direct dependencies, the mathematical model is allowed to find windows of parallelism, ensuring delivery within the schedule.

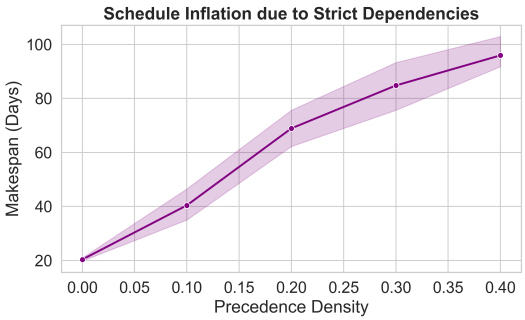

Figure 12: Deadline inflation due to precedence.

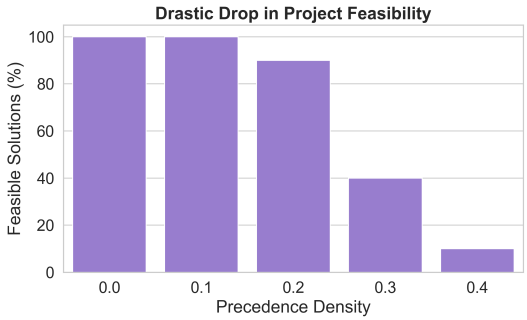

Figure 13: Project feasibility by precedence density.

4.6. Trade-off between Technical Proficiency and Behavioral Alignment

The final stage of the analysis explores the trade-off relationship between technical productivity gain (ET) and losses generated by behavioral misalignment (EC). Figure 14 confirms the traditional behavior expected in the literature: when we maintain the team with a stable psychological profile, increasing technical seniority (higher ET) results in consistent reductions in the *makespan*.

However, the main contribution of this model lies in the juxtaposition of these two metrics, revealed in Figure 15. The mapping of the intersection point of the curves demonstrates an important result: a Junior developer ($ET = 0.0$) suited to the behavioral requirements of the task will be able to execute it in less time than a Senior developer ($ET = 1.0$) operating in the Critical or Extreme error zones. This scenario suggests that high technical qualification alone is not enough to compensate for the delays generated by a strong incompatibility of profiles in the team. Consequently, *squad* formation practices that incorporate psychological mapping cease to be merely a human resources differential and come to mean a real possibility of generating greater operational efficiency.

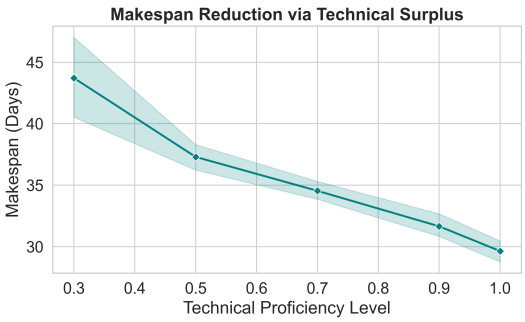

Figure 14: Makespan Reduction via Technical Surplus.

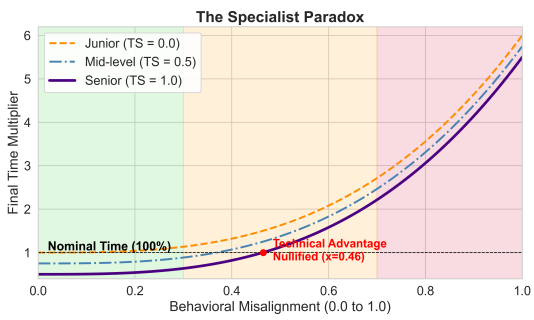

Figure 15: Relation between Technical Surplus and Behavioral Noise.

5. Conclusions and Future Work

This paper presented a mathematical modeling for the Software Project Scheduling Problem (SPSP) that seeks to complement traditional approaches by incorporating not only technical constraints but also behavioral variables. Through a MILP model and the calibration of a cubic penalty function, the results indicate that integrating the psychological

compatibility of the team into task allocation contributes to the generation of more accurate and realistic schedules.

The computational experiments generated quantitative results applicable to tactical project management. First, it was demonstrated that planning blind to the human profile presents significant deviations, with the Extreme friction scenario inflating the *makespan* to levels that surpass the structural capacity of the *sprints*. Second, the model mathematically simulated Brooks's Law, identifying the saturation point where increasing the team stops accelerating deliveries. Finally, a relevant socio-technical dynamic was observed: a Junior developer aligned with the task's profile can finish it in less time than a Senior subjected to a high level of cognitive noise.

As guidelines for future research, the expansion of the mathematical model is proposed to cover more complex operational dynamics. Among the mapped improvements, topological flexibilization stands out to allow the splitting of extensive tasks across multiple *sprints* and the refinement of precedence constraints, enabling the start of dependent activities within the same agile cycle. Furthermore, it is planned to introduce individual parallelism (one developer working on multiple simultaneous tasks) and the allocation of small teams for a single activity. In the latter case, the modeling should incorporate variables to quantify communication *overhead*, the adaptation curve[2], and intra-team behavioral synergy[3], ensuring that the addition of new members complements the work cell and does not intensify cognitive friction.

Acknowledgements

We acknowledge the support of the Fundação de Amparo à Pesquisa de Minas Gerais (FAPEMIG) and the Pró-Reitoria de Pesquisa da Universidade Federal de Minas Gerais (PRPq/UFMG) through the funding obtained by the Edict PRPq PROBIC/FAPEMIG - 11/2025.

References

Acuña, S. T., Gómez, M., and Juristo, N. (2006). How do personality, team processes, and task characteristics relate to job satisfaction and software quality? *Information and Software Technology*, 48(8):622–636.

Alba, E. and Chicano, F. (2007). Software project management with ga's. *Information Sciences*, 177(11):2380–2401.

Blazewicz, J., Lenstra, J. K., and Rinnooy Kan, A. H. (1983). Scheduling subject to resource constraints: classification and complexity. *Discrete Applied Mathematics*, 5(1):11–24.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley Professional.

Chicano, F., Luna, F., Nebro, A. J., and Alba, E. (2011). Using multi-objective metaheuristics to solve the software project scheduling problem. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, p. 1915–1922.

Costa, M. and Lima, F. (2018). Análise de perfil comportamental: A metodologia profiler como ferramenta estratégica. *Revista Brasileira de Recursos Humanos*, 12(4):45–59.

[2]Refers to the initial time and cognitive effort required for the team to understand the context, code architecture, and specific requirements of a task before reaching their full productivity.

[3]A theoretical metric that evaluates how the combination of different psychological profiles allocated to the same activity affects collaboration, problem-solving dynamics, and the group's joint efficiency.

DeMarco, T. and Lister, T. (1987). *Peopleware: Productive Projects and Teams*. Dorset House Publishing Co.

Dikert, K., Paasivaara, M., and Lassenius, C. (2016). Challenges and success factors for large-scale agile transformations: A systematic literature review. *Journal of Systems and Software*, 119:87–108.

Faria, A. G., Machado, G. L., Reis Jr, J., and Silva, L. (2022). Distribuição de tarefas em uma fábrica de software: Equilibrando entregas e lucro em otimização mono-objetivo. *Universidade Federal de Minas Gerais - UFMG*.

Kniberg, H. and Ivarsson, A. (2012). Scaling agile @ spotify with tribes, squads, chapters and guilds. Technical report, Spotify Engineering.

Kolisch, R. and Padman, R. (1999). An integrated survey of project scheduling. *Omega - The International Journal of Management Science*, 29(3):249–272.

Pritsker, A. A. B., Watters, L. J., and Wolfe, P. M. (1969). Multiproject scheduling with limited resources: A zero-one programming approach. *Management science*, 16(1):93–108.

Ryan, S. M. and Cho, J. (2013). A mathematical programming approach to the software project scheduling problem. *IEEE Transactions on Engineering Management*, 61(1): 145–155.

Schwaber, K. and Sutherland, J. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org, 2017.

Vega-Velázquez, M. Á., García-Nájera, A., and Cervantes, H. (2018). A survey on the software project scheduling problem. *International Journal of Production Economics*, 202: 145–161.